PA2: Story Tree

A choose-your-own-adventure RPG built on a tree

Course	CS 210, Data Structures (C++)
Format	Programming assignment, solo, 100 points
Time	Two weeks. See Canvas for the due date.
Starter	`cs210_pa2_starter.zip`
Also read	`cs210_argv_fileio_reading.pdf` (command line arguments and reading files)
Submission	A link to your public GitHub repository. Nothing else accepted.

1 The big idea

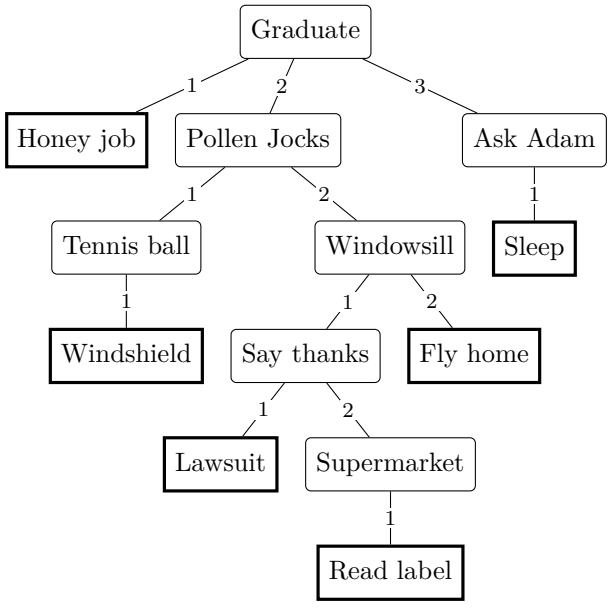
Every choose-your-own-adventure story is a tree.

The first scene is the **root**. Every choice the player can make is a branch to a **child** scene. A scene with no choices is an **ending**, which is a **leaf**. One playthrough is one walk from the root down to a leaf.

You will build the engine that loads a story from a file, lets a player walk through it, and analyzes it. The protagonist has **HP**. Some scenes hurt. If HP hits 0, the game is over before reaching an ending. Scenes can also give **loot** points and **items**, which go into an inventory.

Then you will write your own story. Nobody else has your story, and that is the point.

Here is the sample story, `bee_movie.txt`, drawn as a tree. Numbers on the branches are the choice numbers. Boxes with a thick border are endings.



This story has 13 scenes, 6 endings, and a longest playthrough of 5 choices.

2 Key terms

Term	Meaning
Root	The first scene of the story. The tree has exactly one.
Child	A scene you can reach from another scene by making one choice.
Ending (leaf)	A scene with 0 choices. The story stops there.
Degree	How many children a scene has. In PA2 the degree is 0, 1, 2, or 3. Never more.
Height	The most choices a player can make on any path from the root to an ending. One lone scene has height 0.
Path	The choice numbers taken from the root, written like `2-2-1`.
Pre-order	An order that lists a scene first, then everything under its first choice, then everything under its second choice, and so on. The story file is written in pre-order.
Double pointer	A pointer to a pointer, like `Scene<T>** choices_`. Here it points to an array whose slots are each a `Scene<T>*`.
Exact-size allocation	Allocating exactly as many slots as the data needs, no more. A scene with 2 choices gets 2 slots.
Owns	If an object owns a pointer, that object is responsible for deleting it.
Greedy	An approach that makes the best-looking choice right now and never looks ahead.
Optimal	The best possible answer after considering every option.

3 Game rules

1. The player starts with `START_HP` health and 0 loot.
2. Each time the player **enters** a scene, including the very first one, that scene's HP change is added to their HP and its loot is added to their loot.
3. If the scene has an item, it goes into the inventory. If the player already carries an item with that name, they do not get a second one.
4. If HP is 0 or less after entering a scene, the game is over. This can happen on any scene, not only endings.
5. If the player is still alive and the scene is an ending, they reached an ending.
6. Otherwise the player picks one of the scene's choices and enters that scene.

4 The story file

A story file is plain text. The first line sets the starting HP. Then comes the root scene. Each scene is exactly five lines, followed by its choices. Each choice is a `LABEL` line followed by the whole scene it leads to.

Line	Meaning
`START_HP <n>`	The player's starting health. Appears once, at the top.
`SCENE <text>`	What the player reads when they enter this scene.
`HP <n>`	Added to the player's HP on entering. Negative hurts, positive heals.
`LOOT <n>`	Added to the player's loot on entering. Never negative.
`ITEM <name>`	An item the player picks up here, or `none`.
`CHOICES <n>`	How many choices this scene has. Must be 0 to 3.
`LABEL <text>`	The words on the button for one choice. The scene for that choice comes right after.

The start of `bee_movie.txt`:

```
START_HP 10
SCENE Barry B. Benson graduates. Every bee job is honey. Forever.
HP 0
LOOT 0
ITEM none
CHOICES 3
LABEL Take the honey stirring job
  SCENE You stir honey for the rest of your life. Speedrun complete.
  HP 0
  LOOT 2
  ITEM Stirring Stick
  CHOICES 0
LABEL Fly out with the Pollen Jocks
  SCENE You leave the hive for the first time. The world smells like flowers.
  HP -2
  ...
```

Rules for every story file:

- Indentation is optional. It is only there so humans can read the file. Spaces at the start of a line are ignored. Blank lines are ignored.
- The number of `LABEL` lines under a scene must match its `CHOICES` number.
- You may assume every number in the file is a real number. You do not need to handle `HP banana`.
- A scene with more than 3 choices is an error. Your program must reject the whole file.

5 File and variable guide

This section says what each file holds and what each variable means. It does not say how to use them. That part is yours.

`main.cpp`

Runs the program. PART 0 prints the story file line by line. PART 1 is the game menu. Both are given. PART 1 starts commented out.

Variable	What it means
`argc`	How many words were typed on the command line, including the program name.
`argv`	The words themselves. `argv[1]` is the name of the story file.
`game`	The one `Adventure` object for the whole run.
`choice`	The menu option the user typed.

`Moment.h` (given)

Everything one scene says and does. This is the `T` stored inside each scene in this game.

Variable	What it means
`text_`	What the player reads in this scene. From the `SCENE` line.
`label_`	The button text of the choice that leads into this scene. The root's label is `START`.
`hpChange_`	How much HP changes on entering. From the `HP` line.
`loot_`	How much loot is gained on entering. From the `LOOT` line.
`item_`	The item found here, or `none`. From the `ITEM` line.

`Item.h` (given)

One thing in the inventory. Two items are equal when their names match.

Variable	What it means
`name_`	The item's name, like `Pollen Pack`.

`Node.h` and `List.h` (given)

A working linked list, the same idea as your earlier list work. It is the inventory. It owns every pointer you give it.

Variable or method	What it means
`data` (Node)	A pointer to the value this link holds.
`next` (Node)	The next link, or `nullptr` at the end.
`head_`	The first link in the list, or `nullptr` if the list is empty.
`size_`	How many values are in the list.
`addFront(value)`	Adds a value at the front. The list now owns it.
`deleteFront()`	Removes and deletes the front value.
`search(target)`	True if a value equal to `target` is in the list.
`size()`, `isEmpty()`, `print()`	What they say. `print` writes one value per line.

`Scene.h` (you write parts)

One scene of the story and the doors leading out of it. It works for any `T`.

Variable	What it means
`data_`	A pointer to this scene's payload. In this game the payload is a `Moment`. The scene owns it.
`choices_`	A double pointer. It points to an array of scene pointers. Each slot points to one scene the player can go to next.
`numChoices_`	How many slots `choices_` has. Zero means this scene is an ending.
`MAX_CHOICES`	The most choices any scene may have. It is 3 and cannot change.

`StoryTree.h` (you write parts)

The whole story. It works for any `T` and never mentions `Moment`.

Variable	What it means
`root_`	The first scene, or `nullptr` if no story is loaded. The tree owns the whole story through it.
`scene` (parameter)	The scene where a question starts. The private helpers answer their question for the part of the story below `scene`.

`Adventure.h` and `Adventure.cpp` (you write parts)

The game itself. It loads one story into a `StoryTree<Moment>`, plays it, and analyzes it.

Variable	What it means
`tree_`	The loaded story.
`startHp_`	The number from the `START_HP` line.
`in`	The open story file being read.
`label`	The button text of the choice that led to the scene being read or built.
`hp`	The player's health at this moment.
`loot`	The player's loot so far.
`bag`	The player's inventory, a `List<Item>`.
`path`	The choice numbers taken from the root to reach a scene, like `"2-1"`. Empty for the root.
`bestPath`	Filled in by `bestLoot` with the path of the best run it found.
`numChoices`	How many choices the current scene offers.

Three helpers at the top of `Adventure.cpp` are given and finished.

- `nextLine(in, line)` reads the next non-blank line and removes leading spaces.
- `readField(in, keyword, value)` reads the next line, checks that it starts with `keyword`, and puts the rest into `value`. It prints an error and returns false if the line is wrong.
- `extendPath(path, n)` adds one choice number to a path. `extendPath("2-1", 3)` gives `"2-1-3"`.

Story files

File	What it holds
`bee_movie.txt`	The sample story. Your output must match Section 9 exactly.
`bad_story.txt`	A broken story with a scene that has 4 choices. Your program must reject it.
`my_story.txt`	Your own story. Replace the placeholder.
`CMakeLists.txt`	Builds `adventure` and copies the story files next to it.

6 Getting started

1. Read `cs210_argv_fileio_reading.pdf`.
2. Put the starter files in your repository and build. In CLion, set the Program arguments to `bee_movie.txt`.
3. Run it. PART 0 prints the story file. If you see it, your setup works.
4. Work the TODOs in order: `Scene.h` (1 to 5), `StoryTree.h` (6 to 9), then `Adventure.cpp` (10 to 19).
5. When everything compiles, comment out PART 0 in `main.cpp` and uncomment PART 1. Do not change PART 1.
6. Compare your output with Section 9, one menu option at a time.

7 What to implement

Each TODO in the starter has a comment with the exact output it must print. This section says what each piece must do. The "Think about it" boxes are questions, not answers.

7.1 Scene.h

TODO 1, constructor body. The initializer list is given. Allocate exactly `numChoices_` slots for `choices_`. An ending gets no array at all, so `choices_` stays `nullptr`. Every slot must start as `nullptr`.

TODO 2, destructor. Free everything the scene owns. After a scene is deleted, nothing it owned may still be allocated.

TODO 3 to 5. `isEnding` says whether the scene is an ending. `getChoice` returns the scene behind one slot, or `nullptr` for a bad index. `setChoice` stores a scene in one slot and does nothing for a bad index.

> **Think about it**
>
> What is inside a slot right after `new Scene<T>*[n]`, before you set it? Why does that matter for the destructor?
>
> A scene owns its payload, its array, and something else. What is the something else? Does the

> order you free them in matter?

7.2 StoryTree.h

TODO 6 to 8. The private `countScenes`, `countEndings`, and `height` helpers. Each must be recursive. Each answers its question for the part of the story that starts at `scene`. An empty story (`nullptr`) has 0 scenes, 0 endings, and height -1.

TODO 9, `clear`. Free the whole story and leave `root_` as `nullptr`.

> **Think about it**
>
> If you already know the answer for each child, how do you get the answer for the parent? Those three helpers differ in only one or two lines.

7.3 Adventure.cpp: loading

TODO 10, `load`. Open the file, read `START_HP`, build the story with `readScene`, and give the root to `tree_`. Return false on any failure.

TODO 11, `readScene`. Read one scene and every scene below it, and return it. Reject a `CHOICES` value below 0 or above `MAX_CHOICES` with the exact error in the starter. On any failure, return `nullptr` and leak nothing.

> **Think about it**
>
> The file is written in pre-order. Your function reads a scene, then needs that scene's children. Who already knows how to read a whole scene with everything below it?
>
> If the third child fails to load, the first two are already built. Where do they live now, and who will free them?

7.4 Adventure.cpp: playing

TODO 12, `askChoice`. Read a whole line with `std::getline`. Accept only `1` to `numChoices`. Anything else, including `banana`, 7, or an empty line, prints the reprompt and asks again. Return 0 if input ends.

TODO 13, `play`. Walk the story from the root, following the game rules and the print order in the starter comment. The inventory is a `List<Item>`. Use `search` to avoid duplicates.

7.5 Adventure.cpp: analyzing

TODO 14 and 15. `printStats` prints the three numbers from `tree_`. `printSpoilers` starts the ending list.

TODO 16, `printEndings`. Recursive. Print every ending with the path that reaches it, choice 1 before choice 2 before choice 3.

TODO 17, `greedyRun`. The greedy advisor. At every scene it moves to the choice whose next scene has the most loot. Ties go to the lower choice number. It never looks further ahead and it

never checks HP before choosing. It stops when it dies or reaches an ending.

TODO 18, `bestLoot`. Recursive. The most total loot you can collect from `scene` down to an ending without HP reaching 0. Return -1 if every way down dies. Fill `bestPath` with the winning path.

TODO 19, `optimalRun`. The optimal advisor. Use `bestLoot` from the root and print the result.

> **Think about it**
>
> Greedy is a loop. Optimal is recursion. Why can greedy get away with a loop?
>
> In `bestLoot`, a child that returns -1 is a dead end. What should a scene return if all of its children are dead ends? What if the scene itself kills you?

8 Rules

Breaking a rule loses the points for that part.

- `choices_` stays `Scene<T>**` with exact-size allocation. No `std::vector`, no `std::array`, no fixed array like `Scene<T>* choices_[3]`.
- No STL containers anywhere in your code. `std::string`, `<fstream>`, and `<iostream>` are fine.
- Do not change any given file, signature, or member name. Do not change PART 1 of `main.cpp`.
- `Scene` and `StoryTree` stay templates and never mention `Moment`.
- The required recursive functions must be recursive: TODO 6, 7, 8, 11, 16, and 18.
- No memory leaks, including when a bad story is rejected.

9 Expected output

Your output must match these exactly, character for character. When you type, your input shows after the prompt.

9.1 Bee Movie: stats, spoilers, and advisors

Input: `2`, `3`, `4`, `5`.

```
$ ./adventure bee_movie.txt
Story loaded.

==== MENU ====
1. Play
2. Story stats
3. Spoiler guide
4. Greedy vs optimal
```

```
5. Quit
Choose: 2
Scenes: 13
Endings: 6
Longest playthrough: 5 choices

==== MENU ====
1. Play
2. Story stats
3. Spoiler guide
4. Greedy vs optimal
5. Quit
Choose: 3
Path 1: You stir honey for the rest of your life. Speedrun complete.
Path 2-1-1: You hit the windshield at 60 miles per hour.
Path 2-2-1-1: You win the lawsuit. Bees get their honey back.
Path 2-2-1-2-1: The label says made by bees. You start a revolution.
Path 2-2-2: Back at the hive. Nobody believes your story.
Path 3-1: You sleep through the whole movie.

==== MENU ====
1. Play
2. Story stats
3. Spoiler guide
4. Greedy vs optimal
5. Quit
Choose: 4
Greedy advisor
  Path: 2-1-1
  Loot: 10
  Result: died
Optimal advisor
  Path: 2-2-1-2-1
  Loot: 15
  Result: reached an ending

==== MENU ====
1. Play
2. Story stats
3. Spoiler guide
4. Greedy vs optimal
5. Quit
Choose: 5
Goodbye!
```

Greedy grabbed the fuzzy ball for 6 loot and flew straight into a windshield. Optimal looked at every path and found 15 loot with HP to spare.

9.2 Bee Movie: a winning playthrough

Input: `1`, `2`, `2`, `1`, `2`, `1`, `5`.

```
$ ./adventure bee_movie.txt
```

```
Story loaded.

==== MENU ====
1. Play
2. Story stats
3. Spoiler guide
4. Greedy vs optimal
5. Quit
Choose: 1
----------------------------------------
Barry B. Benson graduates. Every bee job is honey. Forever.
HP: 10  Loot: 0
[1] Take the honey stirring job
[2] Fly out with the Pollen Jocks
[3] Ask Adam for advice
Your choice: 2
----------------------------------------
You leave the hive for the first time. The world smells like flowers.
You picked up: Pollen Pack
HP: 8  Loot: 3
[1] Chase the giant fuzzy ball
[2] Rest on a windowsill
Your choice: 2
----------------------------------------
A florist named Vanessa spares your life.
You picked up: Thank You Note
HP: 9  Loot: 5
[1] Say thank you out loud
[2] Fly home and keep quiet
Your choice: 1
----------------------------------------
Bees are not supposed to talk to humans. You do it anyway.
HP: 9  Loot: 8
[1] Sue the human race
[2] Visit a supermarket
Your choice: 2
----------------------------------------
Shelves of honey. Bees did all of the work.
You picked up: Price Tag
HP: 8  Loot: 10
[1] Read the label
Your choice: 1
----------------------------------------
The label says made by bees. You start a revolution.
HP: 8  Loot: 15
THE END.
Final HP: 8  Final loot: 15
Inventory (3 items):
  - Price Tag
  - Thank You Note
  - Pollen Pack

==== MENU ====
1. Play
```

```
2. Story stats
3. Spoiler guide
4. Greedy vs optimal
5. Quit
Choose: 5
Goodbye!
```

9.3 Bee Movie: bad input and a game over

Input: 1, 2, banana, 7, 1, 1, 5.

```
$ ./adventure bee_movie.txt
Story loaded.

==== MENU ====
1. Play
2. Story stats
3. Spoiler guide
4. Greedy vs optimal
5. Quit
Choose: 1
----------------------------------------
Barry B. Benson graduates. Every bee job is honey. Forever.
HP: 10  Loot: 0
[1] Take the honey stirring job
[2] Fly out with the Pollen Jocks
[3] Ask Adam for advice
Your choice: 2
----------------------------------------
You leave the hive for the first time. The world smells like flowers.
You picked up: Pollen Pack
HP: 8  Loot: 3
[1] Chase the giant fuzzy ball
[2] Rest on a windowsill
Your choice: banana
Please type a number from 1 to 2.
Your choice: 7
Please type a number from 1 to 2.
Your choice: 1
----------------------------------------
A tennis ball nearly flattens you. Totally worth it.
You picked up: Tennis Fuzz
HP: 2  Loot: 9
[1] Ride a windshield home
Your choice: 1
----------------------------------------
You hit the windshield at 60 miles per hour.
HP: -3  Loot: 10
Your HP hit 0. GAME OVER.
Final HP: -3  Final loot: 10
Inventory (2 items):
  - Tennis Fuzz
```

```
  - Pollen Pack

==== MENU ====
1. Play
2. Story stats
3. Spoiler guide
4. Greedy vs optimal
5. Quit
Choose: 5
Goodbye!
```

9.4 Bad runs that must fail

These are tested. Each must print exactly one line and exit with code 1. The usage and could not open checks are already in the starter. The `bad_story.txt` check is yours, in `readScene`. It must also leak nothing.

```
$ ./adventure
Usage: ./adventure <story_file>
(exit code 1)

$ ./adventure nope.txt
Error: could not open file nope.txt
(exit code 1)

$ ./adventure bad_story.txt
Error: scene "The hallway has four doors." has 4 choices. The limit is 3.
(exit code 1)
```

10 Your own story

Replace `my_story.txt` with an original story. It must load and play in your program, and it must have:

- at least **15 scenes**,
- at least **4 endings**,
- a height of at least **5**, meaning at least one path of 5 or more choices,
- at least **one scene with 3 choices**.

Any genre. Any protagonist. Make some scenes hurt. Make greedy look bad. Use your own words.

11 Reflection questions

Answer in `REFLECTION.md` at the root of your repository. Answers must refer to your own code and your own story. Generic answers get no credit.

1. For every function you wrote (TODO 1 to 19), explain in your own words what it does and give its time complexity in terms of n, the number of scenes. One short paragraph each.
2. Why does each scene allocate exactly `numChoices` slots instead of declaring `Scene<T>* choices[3]`? What does an ending hold in `choices_`, and why?
3. Walk through your `Scene` destructor. What gets deleted, and in what order? What would go wrong if `choices_` were deleted first?
4. Show the path greedy took and the path optimal took on **your** story, with the loot for each. Explain why greedy lost. If greedy did not lose, change your story until it does, and explain what you changed.
5. `bestLoot` tries every path, yet it runs in $O(n)$ time. Explain why it is not exponential, using what you know about paths and leaves in a tree.
6. Which `List` methods does your game use, and why? What does one `search` cost as the bag grows?

12 Grading

Part	Points
`Scene.h`: exact-size allocation, slots start empty, accessors (TODO 1, 3, 4, 5)	10
Recursive loader, including the 3-choice limit (TODO 10, 11)	15
Play: entering scenes, HP, loot, inventory, game over, reprompt (TODO 12, 13)	10
Count scenes, count endings, height, 4 points each (TODO 6, 7, 8, 14)	12
Spoiler guide with paths (TODO 15, 16)	8
Memory: destructor and `clear`, no leaks (TODO 2, 9)	5
Greedy advisor (TODO 17)	5
Optimal advisor (TODO 18, 19)	10
Your own story meets every minimum	10
`bad_story.txt` rejected with the exact message, exit code 1, no leaks	5
`REFLECTION.md`	10
Total	**100**

13 Before you submit

- ☐ PART 0 is commented out and PART 1 is uncommented and unchanged.
- ☐ All three Bee Movie runs in Section 9 match exactly.
- ☐ All three bad runs print the exact line and exit with code 1.
- ☐ `choices_` is still `Scene<T>**` with exact-size allocation.
- ☐ `Scene.h` and `StoryTree.h` never mention `Moment`.
- ☐ `my_story.txt` is your own and meets every minimum.
- ☐ `REFLECTION.md` answers all six questions about your own code.
- ☐ Your repository is public.
- ☐ You submitted a link to your repository, nothing else.